- Chassisdesigns (Fotos/Renderings) + Befestigungen (Fotos)
- Lenkmechanismus (mit Fotos/Darstellungen)
- Antrieb/Getriebe (Fotos) + Geschwindigkeitsberechnungen (kleine Tabelle)

Alle für den Wettbewerb bzw. für einen Nachbau notwendigen Code-Dateien sind online unter [Link GitHub] verfügbar. Aufgrund der Komplexität und Länge des Codes (> 3000 Zeilen) sowie der, leider nicht mehr so weit verbreiteten, verwendeten Programmiersprachen C/C++ (und Assembler) wird in diesem Kapitel der Dokumentation die Funktionsweise unseres modularen Codes vorzugsweise mit Grafiken und Programmablaufplänen (nach DIN 66001) erklärt.

Der Code für die Auswertung der Kameradaten, welcher auf dem Raspberry Pi 5 ausgeführt wird, wurde vom letzten Jahr vollständig übernommen und nur etwas modifiziert, um dem neuen Regelwerk zu entsprechen. Wenn das Programm durch den Autostart startet, wartet es nun nach dem Senden des Bereitschaftssignals über UART auf eine Antwort, welche die Bildauswertung mit anschließender Objektdatenübertragung an den Teensy 4.1 nach Drücken des Starttasters startet (siehe Abb. 3.1):

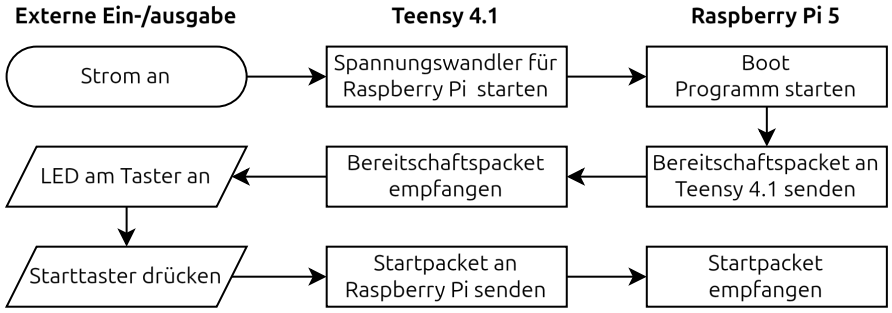


Abbildung 3.1: Programmablaufplan zum Start von der Objekterkennung

Die Objekterkennung geschieht auf dem KI-Beschleunigungsmodul Hailo-8, welches per PCIe 3.0 x 1 mit dem Raspberry Pi 5 verbunden ist. Der Kommandozeilenbefehl

```
rpicam-hello --framerate 50 --hflip --vflip --post-process-file
/home/wro/Documents/hailo_yolo11s.json -n -t 0 -v --width 1296 --height 1296
```

lädt das in der Datei yolo11s.hef gespeicherte KI-Modell auf das Beschleunigungsmodul, konfiguriert die Weitwinkelkamera und führt die Bildauswertung mit einer Rate von 50 Bildern pro Sekunde durch.
Die Konsolenausgabe dieses Befehls, welche die Daten der erkannten Objekte enthält, wird durch eine sogenannte Pipe abgefangen und vom weiteren Programm gelesen, welches die Objektdaten aus der Ausgabe heraussucht und in ein einfaches binäres Format für die serielle Übertragung packt und an den Teensy 4.1 sendet.

Das Hauptprogramm unseres autonom fahrenden Roboterfahrzeuges ist ähnlich wie im letzten Jahr aufgebaut; es wurde teilweise direkt übernommen und nur auf die neue Steuerungseinheit angepasst. Viele Sensorprogramme wurden von uns aber modularisiert und in eigene Sensorbibliotheken verpackt, welche sich im Ordner Teensy/lib/ befinden. Diese Bibliotheken bestehen jeweils mindestens aus einer Headerdatei (.hpp) mit Erläuterungen zur Funktion und der korrekten Benutzung und einer Quelldatei (.cpp) mit dem hinter den Funktionen der Headerdatei liegenden Quellcode. Teilweise sind diese Bibliotheken nur optimierte Versionen von bestehenden Bibliotheken, teilweise wurden sie von uns auf Grundlage von Datenblättern neu geschrieben.
Alle zum Wettbewerb/Fahrzeug spezifischen Funktionen und Strukturen werden in der Datei Teensy/src/car.hpp erläutert, der Quellcode zu ihnen befindet sich in der Datei Teensy/src/car.cpp, zusammen mit den Pin-Definitionen und einigen Hardware-Konfigurationen. Die Konfigurationen von Abstandssensoren und Gyro-Sensor, welche während der Kalibrierung ermittelt werden, sind menschenlesbar auf der eingesteckten SD-Karte abgespeichert, damit die automatisch ermittelten Ergebnisse der Kalibrierung möglichst einfach und schnell auch von außen verändert werden können.